STUDY COACH AI Week 5:
Flow and Architecture

Team Members :
- **Hussein Mohamed Abdel Fattah**
- **Khaled Sherif Mohamed**
- **Sondos Mohamed Hussein Afify**
- **Mohamed Tamer Mohamed**

1. Introduction

This week focuses on translating the Study Coach AI concept into a clear interaction design and technical architecture.

The main objectives are to design the chatbot conversation flow, define the system architecture, evaluate the proposed interface using Shneiderman's Eight Golden Rules, and make the first model call through the team's own backend.

Study Coach AI is an educational assistant designed to help university students understand Human-AI Interaction (HAI) course materials, ask course-related questions, receive explanations at different levels of detail, and practice their understanding.

The initial knowledge scope focuses on HAI Chapters 1 and 2, using approved course materials.

The system considers three user personas developed in the previous week:

- **Mariam Hassan:** A novice learner who needs simple language, clear instructions, and suggested questions.
- **Omar Khalid:** A student who values reliable answers, source references, and the ability to verify information.
- **Amina Ahmed:** An advanced user with low vision who benefits from accessible, structured content and adjustable presentation.

The design aims to provide a simple, accessible, and user-controlled learning experience while helping students understand the sources and limitations of AI-generated answers.

Implementation Status: This document distinguishes between proposed design decisions and implemented features. A feature will only be marked as implemented when it can be demonstrated in the working prototype.

---

2. Conversation Flow Design

2.1. Purpose of the Conversation Flow

The conversation flow describes how students interact with Study Coach AI, starting from the greeting message until they receive assistance or complete their learning task.

It defines the main interaction paths, quick replies, error recovery, fallback behavior, and human handover.

The purpose is to make the chatbot interaction clear, predictable, and easy to use for students with different levels of technical experience.

2.2. Greeting Message

When a student opens Study Coach AI, the system displays a welcoming message explaining its main capabilities.

Proposed Greeting:

"Hello! Welcome to Study Coach AI.

I can help you understand HAI course materials, explain difficult concepts, answer course-related questions, and practice what you have learned.

What would you like to do today?"

The greeting introduces the chatbot's purpose and provides clear options to help students begin interacting with the system.

2.3. Main Conversation Paths

Study Coach AI provides four proposed main interaction paths.

Path 1: Ask a Question

The student asks a question related to the HAI course.

Example:

"What is Human-AI Interaction?"

System Behavior:

1. The student enters a question.
2. The system validates the input.
3. The backend retrieves relevant passages from the approved course materials, when retrieval is available.
4. The backend constructs a prompt containing the question and relevant context.
5. The prompt is sent to the language model.
6. The system receives the generated response.
7. The answer and its supporting source reference are displayed to the student when available.

Path 2: Explain a Topic

The student selects a topic and chooses a preferred explanation style.

Available Explanation Modes:

- Hint.
- Simple Explanation.
- Step-by-Step Explanation.
- Detailed Explanation.

Example:

The student asks:

"Explain the Inform principle."

Then selects:

"Simple Explanation."

The system provides an explanation using simpler language and relevant examples.

This path supports the Allow Control principle by allowing students to choose how they want information to be presented.

Path 3: Practice Questions

The student chooses to practice a course topic or chapter.

Example:

"Give me five questions about Chapter 1."

Proposed System Behavior:

1. The student selects a chapter or topic.
2. The system presents a practice question.
3. The student submits an answer.
4. The system provides feedback.
5. The system explains the correct answer when appropriate.
6. The student can review mistakes or continue practicing.

This feature is proposed and should only be marked as implemented if a working practice feature is available.

Path 4: Get Help or Report a Problem

The student requests additional help or reports an issue with an AI-generated answer.

Examples:

- "This answer is incorrect."
- "I don't understand this explanation."

- "I need help from my instructor."

Proposed System Behavior:

1. The student selects the feedback or help option.
2. The system identifies the type of problem.
3. The system offers clarification or an appropriate next step.
4. If the issue requires an authoritative response, the system directs the student to the instructor, teaching assistant, or official course communication channel.

2.4. Quick Replies

Quick replies are predefined options that allow students to perform common actions without typing complete messages.

They reduce typing effort and make the interface easier to use, especially for novice users such as Mariam.

Welcome Screen Quick Replies:

- Ask a Question.
- Explain a Topic.
- Practice Questions.
- Get Help.

After-Answer Quick Replies:

- Explain More Simply.
- Give Me a Hint.
- Show Step-by-Step.
- Ask Another Question.
- Report This Answer.

These options should remain consistent across the interface.

Practice and feedback options should be labelled as planned until their functionality is implemented.

2.5. Error Recovery

Error recovery describes how the system helps users continue when something goes wrong.

The chatbot should provide clear error messages and useful recovery actions instead of simply stopping the conversation.

Error Condition	Proposed System Message	Recovery Action

Empty input	Please enter a question before sending.	Keep the user on the chat screen.
Unclear question	I am not sure what you mean. Could you rephrase your question?	Try Again / Choose a Topic.
Network failure	I could not connect to the AI service. Please try again.	Retry the request.
Backend error	Something went wrong while processing your request.	Try Again.
Insufficient course information	I could not find enough information in the available course materials.	Try another topic or check official materials.

The system should explain the problem in simple language and help the student recover without restarting the entire interaction.

2.6. Fallback Message

A fallback message is displayed when the chatbot cannot provide a reliable answer.

This may happen when:

- The question is outside the available HAI course materials.
- The retrieved course passages do not contain enough information.
- The system cannot identify relevant supporting evidence.

Proposed Fallback Message:

"I'm sorry, but I couldn't find enough information in the available HAI course materials to answer this question reliably.

You can try rephrasing your question, choose another course topic, or check the official course materials."

The fallback behavior helps prevent unsupported answers and communicates the limitations of the AI system.

The chatbot should not invent information or display a source that does not support its answer.

2.7. Human Handover

Human handover occurs when a student's request requires assistance from a real person or an official source.

Examples include:

- Questions about grades.
- Enrollment or registration problems.
- Official academic deadlines.
- Personal academic cases.
- Questions requiring an instructor's judgment.
- Unresolved questions outside the available course materials.

Proposed Human Handover Message:

"This issue requires assistance from your course instructor or teaching assistant.

Please use the official university or course communication channel to request assistance."

For the initial design, human handover may be implemented as a referral to an official communication channel.

The team should not claim that the chatbot automatically contacts an instructor unless this functionality has actually been implemented and tested.

2.8. Conversation Flow Diagram

The following diagram represents the proposed interaction flow.

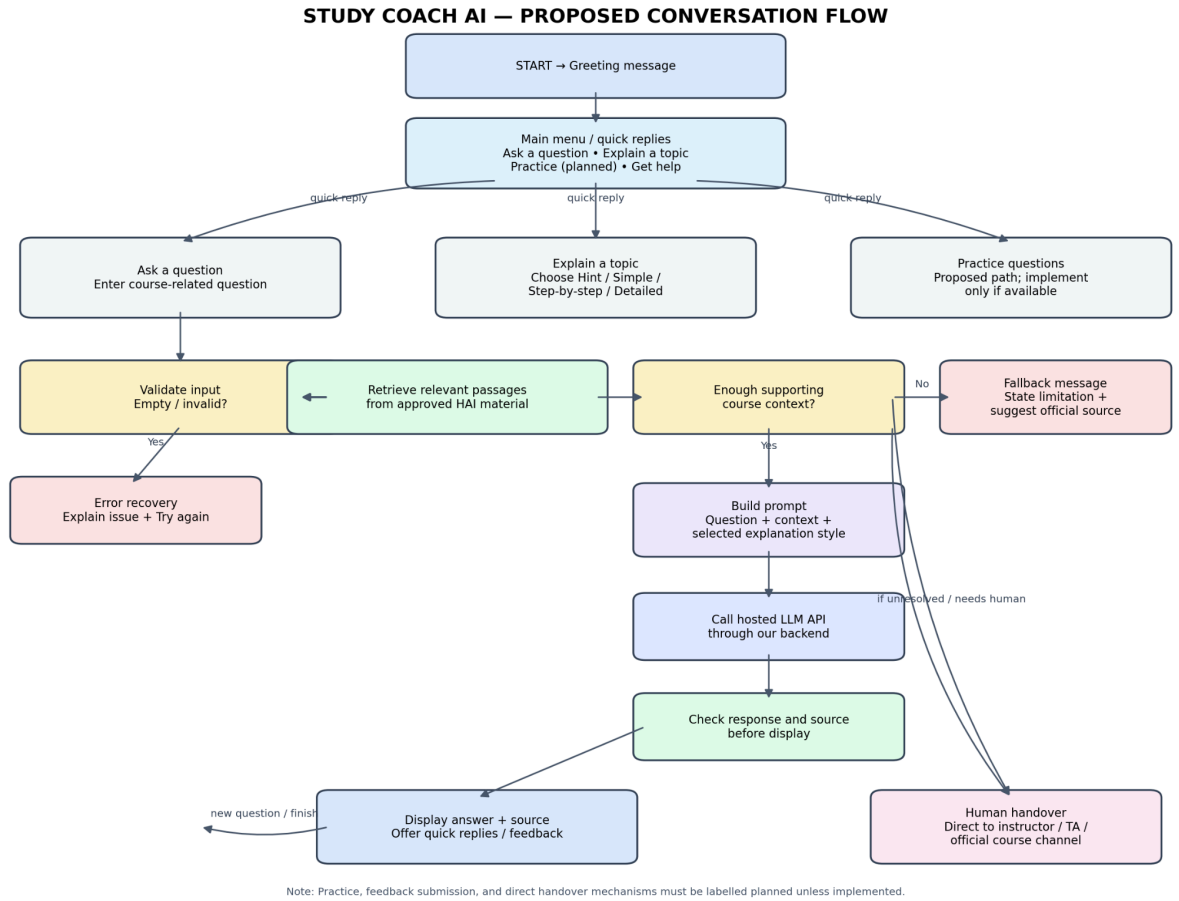


Figure 1. Proposed Study Coach AI Conversation Flow.

The diagram represents the intended design. Features that have not been implemented should remain labelled as planned.

---

3. System Architecture

3.1. Architecture Overview

The proposed Study Coach AI architecture consists of five main components:

1. User Interface.
2. Backend.
3. Language Model.
4. Knowledge Source.
5. Safeguards.

These components work together to receive student questions, retrieve relevant course information, generate responses, and display answers with supporting sources.

The architecture separates the user interface from the backend processing and AI model communication.

3.2. Main System Components

A. User Interface

Technology: Streamlit

The user interface is the part of the system that students interact with directly.

In the current prototype, Streamlit is used to:

- Display the chatbot interface.
- Receive student questions.
- Provide suggested questions through quick-reply buttons.
- Allow students to select different explanation modes.
- Display AI-generated responses.
- Provide a clear conversation option.
- Allow users to provide feedback on responses.

The interface is designed to provide a simple and accessible experience for students with different levels of knowledge.

B. Backend

Technology: Python and FastAPI

The backend manages communication between the user interface and the language model.

In the current prototype, the backend is responsible for:

1. Receiving student questions through the `/chat` endpoint.
2. Validating incoming requests.
3. Preparing the system instructions and user prompt.
4. Sending requests to the local language model through Ollama.
5. Receiving and processing model responses.
6. Returning the generated response to the user interface.
7. Handling errors that may occur during request processing or model communication.

The backend acts as the central processing component of the prototype.

C. Language Model

Technology: Ollama with Llama 3.2 (3B)

The current prototype uses Ollama to run the Llama 3.2 3B language model locally.

The backend sends the student's question and the selected explanation mode to the model.

The model generates a natural-language response, which is returned to the backend and then displayed in the interface.

Running the model locally avoids the need for a hosted LLM API key in the current prototype.

D. Knowledge Source

Planned Knowledge Source: Approved HAI Course Materials

The intended knowledge source includes:

- HAI Chapter 1.
- HAI Chapter 2.
- Other approved course materials, if available.

The proposed design uses a Retrieval-Augmented Generation (RAG) approach.

RAG will retrieve relevant passages from the approved course materials and provide them to the language model as context.

Current Implementation Status:

The current prototype does not yet include document retrieval or RAG integration. Therefore, the model currently generates responses based on its existing knowledge and the instructions provided in the prompt.

Connecting the approved HAI materials is a planned next step to make answers more closely grounded in the course content.

E. Safeguards

Safeguards are mechanisms designed to reduce errors and improve the reliability of the system.

The current prototype and planned design include the following safeguards:

Safeguard	Purpose	Status
Input Validation	Check incoming requests and prevent invalid input.	Implemented
Error Handling	Handle backend and model communication errors.	Implemented
Explanation Modes	Allow students to choose how detailed the explanation should be.	Implemented
Source Checking	Verify that displayed references support the generated answer.	Planned
Insufficient Context Handling	Avoid presenting unsupported answers as reliable when course information is unavailable.	Planned
API Key Protection	Protect API credentials using environment variables if a hosted API is introduced.	Not applicable to the current local setup

Privacy-Aware Logging	Avoid collecting unnecessary personal information.	Planned
Human Handover	Direct students to an instructor or an official course source when needed.	Planned

These safeguards distinguish the features currently available in the prototype from those intended for future development.

3.3. System Request and Response Sequence

The proposed system follows these steps:

1. The student enters a question through the Streamlit interface.
2. The interface sends the question and selected explanation style to the backend.
3. The backend validates the input.
4. The backend retrieves relevant passages from the approved HAI course materials, if retrieval is available.
5. The backend constructs a prompt containing the student's question, retrieved context, and selected explanation style.
6. The backend sends the request to the local language model through Ollama.
7. The model generates a response.
8. The backend receives the response and performs the available output and source checks.
9. The interface displays the answer and supporting source information.
10. If the available information is insufficient or an error occurs, the system displays an appropriate fallback or error message.

3.4. System Architecture Diagram

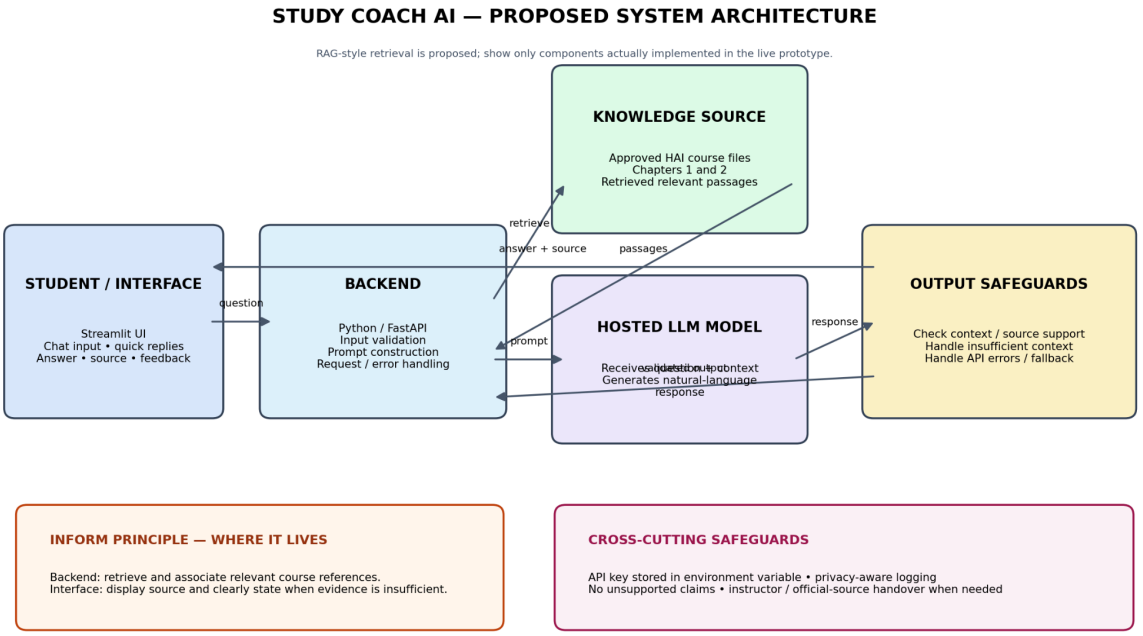


Figure 2. Proposed Study Coach AI System Architecture.

The diagram represents the proposed RAG-style architecture. The final diagram should be updated to match the actual backend implementation and the components used by the team.

3.5. Location of the HAI Principle Feature

The main HAI principle highlighted in this design is **Inform**.

The Inform principle is supported by two main system components.

Backend:

- Retrieves relevant passages from approved course materials.
- Associates the generated response with relevant source information when retrieval is available.
- Identifies situations where sufficient supporting information cannot be found.

User Interface:

- Displays relevant source references alongside answers.
- Shows the chapter or section associated with the information.
- Communicates when the available information is insufficient.

This design helps students verify answers instead of blindly trusting AI-generated information.

The source reference must genuinely support the answer. Displaying a source name alone does not guarantee that the answer is grounded.

3.6 Safeguards and limitations

Safeguard	Purpose
Input Validation	Reject empty or invalid requests before model use.
Source Checking	Ensure displayed references are relevant to the generated answer. .
Insufficient Context Handling	Avoid confident answers when course evidence is missing.
Error Handling	Recover gracefully from network, provider, or backend failures .
API Key Protection	Keep credentials out of source code and public repositories.
Privacy-Aware Logging	Avoid collecting unnecessary personal or sensitive student information.
Human Handover	Refer authoritative or unresolved issues to a real person or official source.

---

4. Evaluation Using Shneiderman's Eight Golden Rules

4.1. Purpose of the Evaluation

Shneiderman's Eight Golden Rules provide a set of design principles that help evaluate the usability and consistency of interactive systems.

These rules are used to review the proposed Study Coach AI interface and identify improvements that can make the interaction easier, clearer, and more predictable for students.

The following checklist evaluates the proposed design. It does not represent the results of formal usability testing.

4.2. Eight Golden Rules Checklist

Rule 1: Strive for Consistency

The interface should use consistent terminology, layouts, and interaction patterns.

Application in Study Coach AI:

- Use the same names for explanation modes throughout the system.
- Keep quick reply options consistent.
- Use a consistent format for displaying answers and source references.
- Maintain predictable navigation between conversation paths.

Rule 2: Enable Shortcuts

The system should provide shortcuts that allow experienced users to perform common actions more quickly.

Application in Study Coach AI:

- Provide quick reply buttons for common actions.
- Allow students to select explanation modes directly.
- Keep frequently used actions accessible without requiring students to type complete commands.
- Allow students to ask another question without repeating the initial setup.

Rule 3: Offer Informative Feedback

The system should provide clear feedback after important user actions.

Application in Study Coach AI:

- Display a loading indicator while processing a question.
- Confirm when a message has been submitted.
- Display an error message if the model API is unavailable.
- Inform students when sufficient course information cannot be found.
- Display relevant source references with answers when available.

Rule 4: Design Dialogues to Yield Closure

The system should provide clear beginnings, progress indicators, and meaningful endings to interaction sequences.

Application in Study Coach AI:

- Start with a greeting and clear options.
- Provide a complete response to the student's question.
- Allow students to review the explanation.
- Offer options such as Ask Another Question or Continue Practicing.
- Provide a clear message when a task or practice session is completed.

Rule 5: Prevent Errors and Handle Them Simply

The system should be designed to prevent common mistakes before they occur.

Application in Study Coach AI:

- Validate empty messages before sending them.
- Check that the user has entered a valid question.
- Protect API credentials using environment variables.
- Validate requests sent to the backend.
- Avoid presenting unsupported information as a verified course answer.

Rule 6: Permit Easy Reversal of Actions

Users should be able to correct mistakes and recover from unwanted actions whenever possible.

Application in Study Coach AI:

- Allow students to edit their questions before resubmitting.
- Provide a Retry option after temporary failures.
- Allow students to request a different explanation style.
- Allow students to ask another question without restarting the application.

Rule 7: Support Internal Locus of Control

Users should feel that they are controlling the system rather than being controlled by it.

Application in Study Coach AI:

- Allow students to choose the explanation level.
- Let students decide whether to request a hint or a detailed explanation.
- Provide optional quick replies instead of forcing a single interaction path.
- Allow students to decide when to continue, retry, or change topics.

This rule is closely related to the Allow Control principle in the project's HAI design.

Rule 8: Reduce Short-Term Memory Load

The system should minimize the amount of information users need to remember while interacting with it.

Application in Study Coach AI:

- Display available actions as quick replies.
- Keep important instructions visible.
- Show source references beside the relevant answer.
- Organize explanations using headings and short sections.
- Avoid requiring students to remember previous commands or complicated instructions.

4.3. Evaluation Summary

The proposed Study Coach AI design addresses the eight Golden Rules through consistent interaction patterns, informative feedback, error recovery, user control, and accessible presentation.

However, these are design-level considerations. Further interface implementation and usability testing are required to determine whether the rules are successfully followed in the working prototype.

---

5. First Model Call Through the Backend

5.1. Objective

The objective of this task is to establish communication between the project's own backend and a Large Language Model (LLM) running locally through Ollama.

Instead of connecting the user interface directly to the model provider, the backend acts as an intermediary between the application and the external AI service.

This approach helps the team manage requests, protect API credentials, validate input, and process model responses.

5.2. Proposed Request Flow

The first model call follows these steps:

1. The user submits a question.
2. The backend receives the question through an API endpoint.
3. The backend validates the input.
4. The backend prepares a request for the selected language model.
5. The backend sends the request to the local Ollama service.
6. The model generates a response.
7. The backend receives and processes the response.
8. The backend returns the result to the client.

5.3. Backend Implementation

Proposed Backend Technology: Python and FastAPI.

The backend should expose an endpoint that accepts a student's question and returns the model's response.

Example endpoint:

```
POST /chat
```

Example request:

```
{
  "question": "What is Human-AI Interaction?"
}
```

Example response structure:

```
{
  "answer": "Human-AI Interaction is the study and design of interactions between people and AI systems."
}
```

The response above is an illustrative example of the expected structure. It must not be treated as an actual model response unless it is produced by the team's running backend.

5.4. Tools and Technologies Used in the Prototype

The initial Study Coach AI prototype uses the following tools and technologies:

Tool	Purpose
Python	Main programming language
FastAPI	Backend API framework
Uvicorn	Runs the FastAPI backend server
Streamlit	Provides the chatbot user interface
Ollama	Runs the language model locally
Llama 3.2 3B	Generates AI responses
Requests	Handles HTTP communication between application components

Pydantic	Validates incoming request data

Model Configuration:

- Backend Framework: FastAPI
- Model Runtime: Ollama
- Model Name: Llama 3.2 3B (default configuration; confirm the installed model before submission)
- Backend Endpoint: `POST /chat`
- Backend Server: Uvicorn
- User Interface: Streamlit

The prototype uses a local language model through Ollama. Therefore, this implementation does not require a paid hosted-model API key.

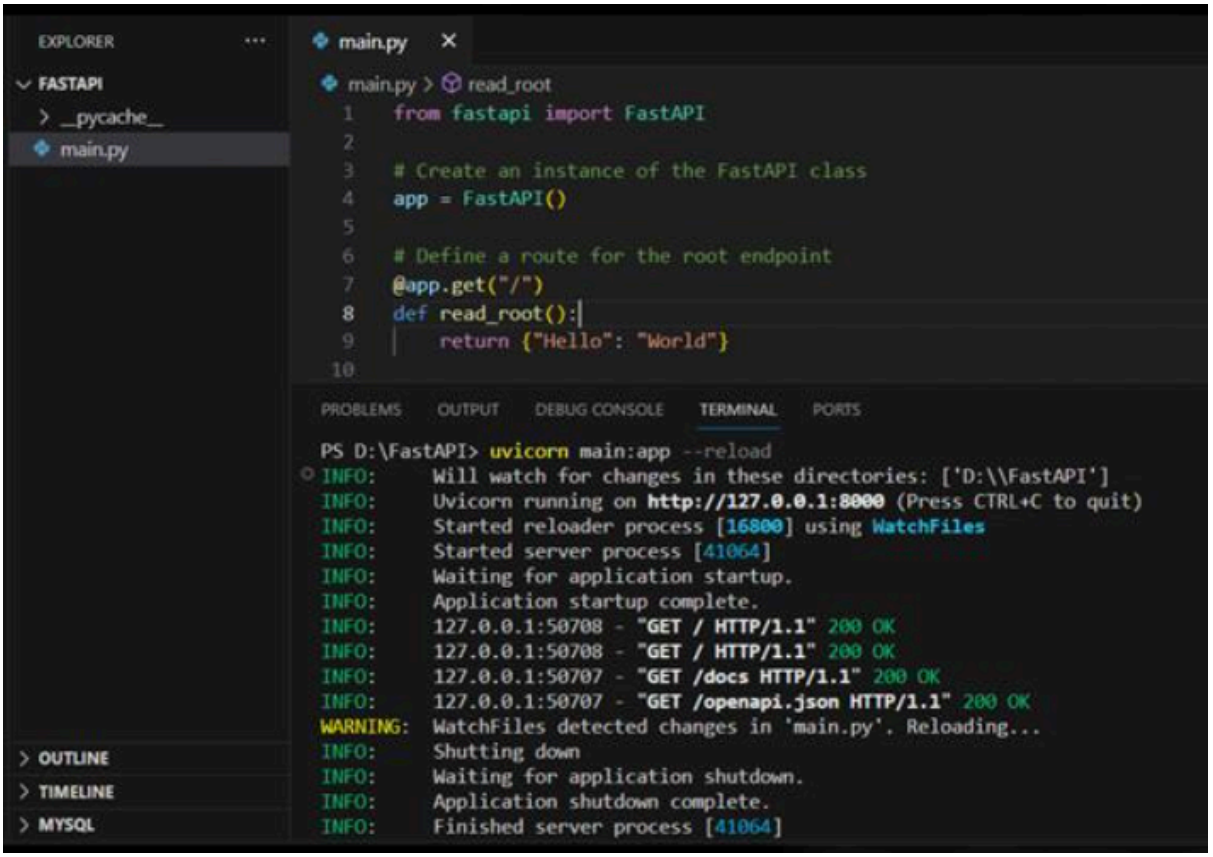
5.5. Prototype Implementation

A preliminary Study Coach AI prototype was prepared to demonstrate the communication between the chatbot interface, backend, and language model.

The prototype includes:

- A Streamlit-based chatbot interface.
- A FastAPI backend that receives student questions.
- An API endpoint (`POST /chat`) for processing questions.
- A local Ollama model for generating responses.
- Four explanation modes: Hint, Simple, Step-by-Step, and Detailed.
- Basic error handling and response display.

The prototype is an initial implementation and does not yet include retrieval from the official HAI Chapters 1 and 2.



5.6. First Model Call Experiment

The first model call is tested by submitting a question through the FastAPI backend.

Test Question:

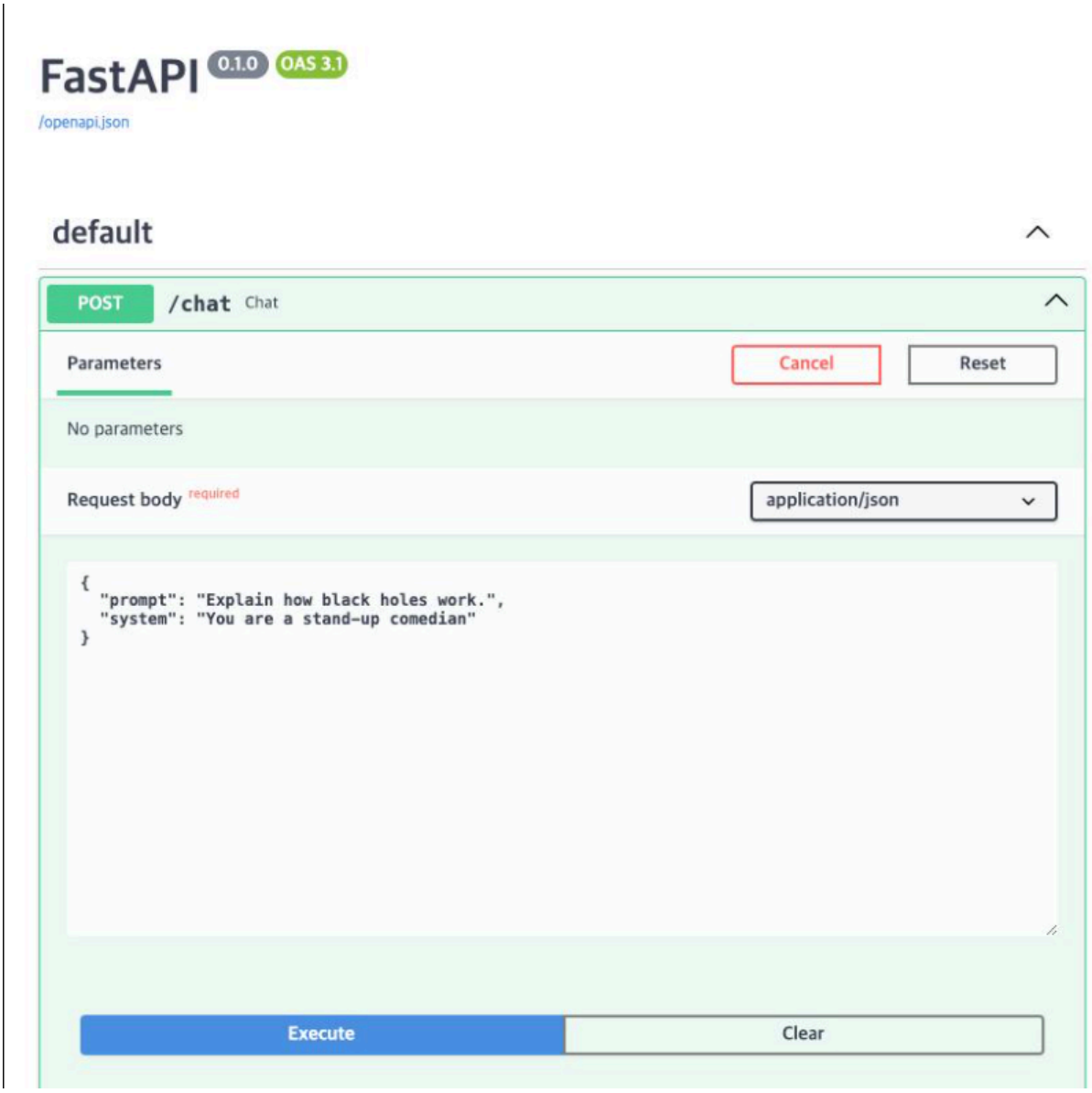
"What is Human-AI Interaction? Explain it in simple language for a university student."

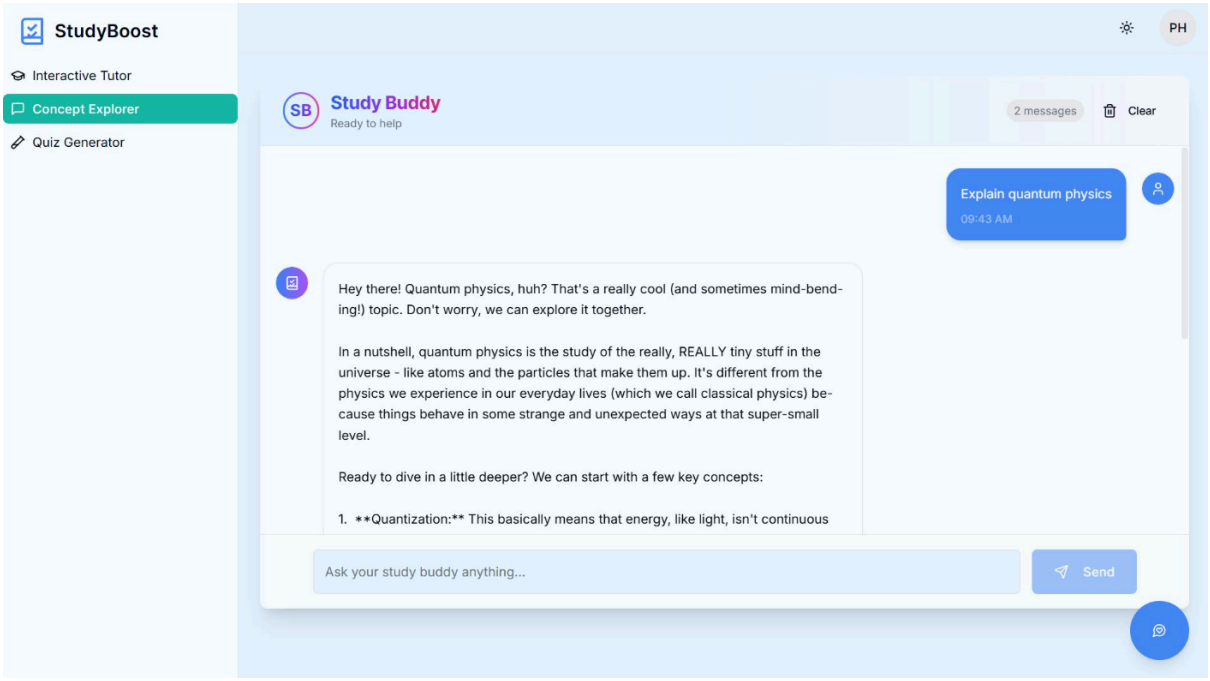
Test Configuration:

- Endpoint: `POST /chat`
- Explanation Mode: Simple
- Model Runtime: Ollama
- Model: [Enter the exact installed model name]
- Test Date: [Enter actual test date]

Actual Experiment Results:

- Connection Status: [Successful / Failed]
- HTTP Status Code: [Enter actual status code]
- Model Response: [Paste the actual response]
- Execution Time: [Enter if measured]
- Error Message: [Enter if any]


FastAPI
0.1.0
OAS 3.1
/openapi.json
default
POST
/chat
Chat
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
{
"prompt": "Explain how black holes work.",
"system": "You are a stand-up comedian"
}
Execute
Clear



Evidence:

The following screenshots should be included after running the prototype:

1. FastAPI backend running successfully.
2. The question submitted through the `POST /chat` endpoint.
3. The actual model response returned by the backend.
4. The Streamlit chatbot displaying the generated answer.

5.7. Prototype Limitations

The current prototype demonstrates the initial backend-to-model communication. However, the following features are not yet implemented:

- Retrieval-Augmented Generation (RAG).
- Retrieval from official HAI Chapters 1 and 2.
- Verified chapter and page references.
- Persistent conversation history.
- Automated instructor handover.

Therefore, the generated responses should be considered general AI-generated explanations rather than answers verified against the official course materials.

---

6. Appendix A — AI Prompt Log

6.1. Purpose

The prompt log documents how AI tools were used during the design and development of Study Coach AI.

It provides a record of the prompts, generated outputs, and the team's review of AI-assisted suggestions.

The team should record only prompts that were actually submitted and should distinguish completed activities from planned activities.

6.2. Prompt Log Entries

Entry 1: Conversation Flow Design

- Tool Used: ChatGPT
- Purpose: To develop the proposed conversation flow for Study Coach AI.
- Prompt Used: "Help me design a complete conversation flow for an educational chatbot called Study Coach AI. Include the greeting, main interaction paths, quick replies, error recovery, fallback messages, and human handover. Consider novice and advanced students and clearly distinguish proposed features from implemented features."
- Output Generated: A structured conversation flow containing the main interaction paths, error conditions, fallback behavior, and human handover.
- Team Review:
  The team reviewed the generated conversation flow and checked whether it covered the required interaction stages, including the greeting, main conversation paths, quick replies, error recovery, fallback messages, and human handover. The proposed flow was also considered in relation to the needs of the three user personas. Some features were kept as planned features for future implementation.
- Status**:** Reviewed and incorporated into the proposed conversation design.

Entry 2: System Architecture Design

- Tool Used: ChatGPT
- Purpose: To organize the proposed technical architecture.
- Prompt Used: "Help me design a system architecture for Study Coach AI using a user interface, Python FastAPI backend, hosted language model API, approved HAI course materials, and safeguards. Explain the responsibilities of each component and show how a student request moves through the system."
- Output Generated: A proposed architecture describing the interface, backend, model communication, knowledge source, and safeguards.
- Team Review:
  The team reviewed the proposed architecture and examined the responsibilities of the user interface, backend, language model, knowledge source, and safeguards. The architecture was adjusted to reflect the technologies selected for the initial prototype, including Streamlit, FastAPI, Ollama, and Llama 3.2 3B. The team also distinguished between implemented components and planned features, particularly RAG and course-material retrieval.

- Status**:** Reviewed and updated to reflect the current prototype architecture.

Entry 3: Shneiderman's Eight Golden Rules

- Tool Used: ChatGPT
- Purpose: To evaluate the proposed interaction design using established usability principles.
- Prompt Used: "Evaluate the proposed Study Coach AI design using Shneiderman's Eight Golden Rules. For each rule, explain its meaning, suggest how it can be applied to the chatbot, and distinguish design recommendations from features that have actually been implemented."
- Output Generated: A checklist connecting the eight rules to the proposed chatbot interface and interaction behavior.
- Team Review**:**
  The team reviewed the generated checklist and evaluated how each of Shneiderman's Eight Golden Rules could be applied to Study Coach AI. The suggested applications were compared with the proposed interface and interaction flow. The checklist was included as a design-level evaluation rather than a claim of completed usability testing.
- Status**:** Reviewed and incorporated into the design evaluation.

Entry 4: Error Recovery and Human Handover

- Tool Used: ChatGPT
- Purpose: To identify possible interaction failures and define appropriate recovery actions.
- Prompt Used: "Identify possible errors in an educational AI chatbot and propose clear recovery messages for empty input, unclear questions, network failures, backend errors, and insufficient course information. Also design a human handover approach for issues that require an instructor or official university support."
- Output Generated: Suggested error messages, recovery actions, fallback behavior, and a referral-based human handover approach.
- Team Review**:**
  The team reviewed the suggested error scenarios and recovery messages to ensure that they were clear and understandable for students. The fallback behavior was designed to avoid presenting unsupported answers as reliable. Human handover was defined as referring students to an instructor, teaching assistant, or official course communication channel rather than claiming that the chatbot automatically contacts them.
- Status**:** Reviewed and incorporated into the proposed error-handling and support design.

- **Entry 5: Backend Prototype Development**
- Tool Used: ChatGPT
- Purpose: To prepare an initial FastAPI backend connected to a local language model.
- Tools and Technologies: Python, FastAPI, Uvicorn, Streamlit, Ollama, and Llama 3.2 3B.

- Prompt Used:
  "Create a preliminary Study Coach AI chatbot prototype using Python and FastAPI for the backend, Streamlit for the user interface, and Ollama with the Llama 3.2 3B model for local language model integration. The application should allow students to submit questions, select different explanation modes (Hint, Simple, Step-by-Step, and Detailed), and receive AI-generated responses through a backend API endpoint. Include input validation and basic error handling, and provide clear instructions for running and testing the application."
- Output Generated**:**
  A preliminary chatbot prototype consisting of a Streamlit user interface and a FastAPI backend. The backend includes a `/chat` endpoint that sends student questions to a local language model through Ollama. The prototype also includes explanation modes and basic error handling.
- Team Review:
  The team reviewed the generated project structure and code, focusing on the separation between the Streamlit interface and FastAPI backend, the `/chat` endpoint, the Ollama connection configuration, and the available explanation modes. The implementation approach was checked against the Week 5 objective of making a model call through the team's own backend. Actual execution and functional testing are still required to confirm successful model communication.
- Status: Prepared – The preliminary prototype code and setup instructions have been generated. Successful execution and testing have not yet been confirmed.

6.3. Prompt Log Documentation Note

The prompt log supports transparency and traceability during AI-assisted development.

AI-generated suggestions are treated as supporting material rather than proof of successful implementation.

The team remains responsible for checking generated content, testing code, verifying technical behavior, and documenting actual changes.

---

7. Challenges, Limitations, and Next Steps

7.1. Expected Challenges

The development of Study Coach AI may involve several technical and usability challenges:

Model Connectivity:

The backend must communicate correctly with the local Ollama service. Model availability, incorrect configuration, or service failures may prevent successful communication.

Answer Reliability:

A language model may generate incorrect or unsupported information. The system should therefore use approved course materials and evaluate whether the generated response is supported by the available evidence.

Source Traceability:

Displaying a source reference is useful only when the cited material genuinely supports the answer. Source retrieval and answer verification require additional testing.

Accessibility:

The interface must consider users with different accessibility needs, including adjustable text size, sufficient contrast, and screen-reader compatibility.

Error Handling:

The application should provide understandable messages and recovery actions when requests fail or course information is insufficient.

7.2. Current Limitations

The current design document describes the intended interaction flow and proposed architecture.

The following capabilities should not be considered completed unless they have been implemented and demonstrated:

- Integration of the prototype interface with the backend and local model: prepared, pending successful execution and verification.
- Retrieval from HAI Chapters 1 and 2: Not implemented.
- Source-grounded response generation: Not implemented.
- Practice-question functionality: Not implemented.
- Automated instructor handover: Not implemented.
- Formal usability and accessibility testing: Not completed.
-

7.3. Next Development Steps

The next development stage will focus on turning the proposed design into a working prototype.

The planned steps are:

1. Confirm the backend framework and selected model provider.
2. Implement the first backend-to-model API call.
3. Test the endpoint and record the actual response.
4. Prepare the approved HAI course materials.
5. Implement document processing and text chunking.

6. Add retrieval to support course-grounded responses.
7. Connect the backend to the user interface.
8. Implement explanation modes and quick replies.
9. Add error handling and fallback behavior.
10. Evaluate the prototype using representative test questions and usability checks.

---

8. Conclusion

Week 5 establishes the interaction design and technical foundation for Study Coach AI.

The proposed conversation flow defines how students ask questions, request explanations, practice course topics, report problems, and receive appropriate support.

The proposed system architecture separates the user interface, backend, language model, knowledge source, and safeguards.

Shneiderman's Eight Golden Rules provide a framework for reviewing the usability of the proposed interaction design.

The first model call is an important implementation milestone because it demonstrates communication between the backend and a locally running language model through Ollama.

Further work is required to implement retrieval from approved HAI materials, connect the complete interface, verify source-grounded answers, and test the system with representative users.

The overall goal is to develop an educational assistant that supports student learning while maintaining transparency, user control, accessibility, and awareness of AI limitations.
